



카테고리 없음

[Euron] Week13_You Only Look Once : Unified, Real-Tme Objec...

yejji 2025. 6. 1. 12:51

0. Abstract

기존의 객체 탐지 방식들은 분류기를 재활용하여 탐지를 수행하였다.

그러나 본 논문에서 제안하는 YOLO는 객체를 탐지할 때 공간적으로 분리된 바운딩 박스와 관련된 클래스 확률로의 회귀 문제로 보고, 단일 신경망이 전체 이미지로부터 바운딩 박스와 클래스 확률을 직접 한 번에 예측 가능하게 한다.

전체 탐지 파이프라인은 1개의 네트워크로 경우에 따라 탐지 성능을 최적화 가능하다.

YOLO는 속도 측면에서 매우 빠르고 mAP도 기존의 방식들과 비교하였을 때 2배 정도 높다.

위치 오류는 많이 생성하나, 배경에 대하여 탐지를 잘못하는 경우는 상대적으로 적다.

YOLO는 객체에 대해 매우 일반화된 표현을 학습하고, 이로 인해 다른 모델들보다 일반화 측면에서 더 나은 성능을 가진다.

1. Introduction

YOLO(You Only Look Once)는 객체 탐지를 단일 회귀 문제로 재구성하여 이미지에서 바로 바운딩 박스 좌표와 클래스 확률을 예측하는 빠르고 간단한 방식의 객체 탐지 시스템이다.

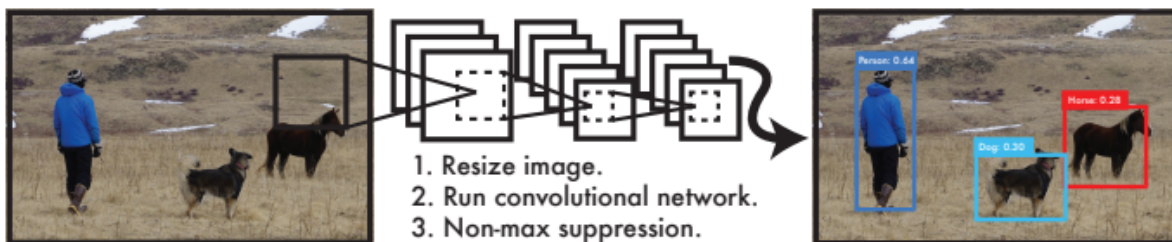
기존의 객체 탐지 방법들은 슬라이딩 윈도우나 Region Proposal을 기반으로 하여 복잡한 파이프라인을 구성하고, 각 구성요소를 따로 학습시켜야 하기 때문에 최적화가 어렵고 속도가 느린 단점이 있었다.

특히나, DPM이 슬라이딩 윈도우 방식을 사용하는데, 이미지 전체에 걸쳐서 여러 개의 위치와 크기에서 객체 분류기를 설정한다.

조금 더 최신 방법인 R-CNN의 경우 Region Proposal을 활용하는데, 바운딩 박스 후보를 설정하고 각각의 영역에 대하여 분류기를 적용한다.

이후 박스를 정제하고 중복된 부분을 제거한 이후 후처리한다.

이와 같이 전체 파이프라인이 복잡하기 때문에, 구성 요소들의 개별적인 학습이 필요하여 최적화 하기에 어렵다.



YOLO의 객체 탐지 시스템

반면에 YOLO는 회귀 문제로 재정의하여 이미지 픽셀들에서 바운딩 박스의 좌표와 클래스 확률을 직접 예측한다.

전체 이미지를 한 번에 처리하고, 단일 컨볼루션 신경망으로 여러 객체의 위치와 클래스 가능성을 동시에 예측한다.

이러한 단순한 구조 덕분에 YOLO는 속도가 매우 빠르다.

또, 탐지를 일종의 회귀 문제로 처리하기 때문에 복잡한 파이프라인이 필요하지 않다.

더불어서, 테스트 시에 새로운 이미지를 넣기만 하면 탐지한 결과 예측이 가능하다.

기본 모델은 GPU 환경에서 초당 45프레임, 경량화된 버전은 초당 150프레임 이상을 처리할 수 있어

실시간으로 영상 분석이 가능하다.

성능 측면에서도 기존의 실시간 시스템보다 **두 배 이상의 평균 정밀도를 기록한다.**

YOLO는 전체 이미지를 기반으로 예측하기 때문에 문맥 정보까지 활용할 수 있어 배경을 객체로 잘못 인식하는 오류가 적다.

또한 자연 이미지로 학습한 후 예술 작품과 같은 새로운 도메인에서도 우수한 성능을 보여, **일반화 성능이 뛰어나다.**

즉, 새로운 도메인이나 예기치 못한 입력에 강한 특성을 가진다는 것이다.

다만, 작은 크기의 객체에 대한 정확한 위치 예측은 어려워 정확도 면에서는 일부 최신 탐지 기법에 비해 뒤처지는 단점도 있다.

YOLO는 코드와 사전 학습된 모델을 모두 오픈소스로 제공하고 있어 다양한 분야에서의 활용 가능성을 넓히고 있다.

2. Unified Detection

YOLO는 객체 탐지의 모든 과정을 1개의 통합된 신경망으로 처리하는 시스템이다.

기존의 방식들이 객체 탐지를 위해 여러 분리된 단계, 예를 들어 Region Proposal, 분류, 위치 조정 등의 과정을 거치지만,

YOLO는 전체 이미지로부터 특징을 추출하여 각각의 바운딩 박스를 예측하고,

전체 이미지와 그 안의 모든 객체들을 전역적으로 고려한다.

이를 통해 YOLO는 실시간 속도로 동작하면서도 높은 평균 정밀도(mAP)를 유지할 수 있다.

YOLO는 입력 이미지를 $S \times S$ 그리드로 분할한다.

객체의 중심이 포함된 그리드 셀이 해당 객체를 예측할 책임을 가진다.

각 셀은 B개의 바운딩 박스와 각 박스에 대한 confidence score를 예측하는데,

이때 confidence는 셀이 객체를 포함할 확률과 예측한 바운딩 박스가 실제 객체와 얼마나 겹치는지를 의미하는 IOU를 곱한 값이다.

바운딩 박스는 (x, y, w, h, confidence)의 총 5개 값으로 구성되며, x와 y는 셀 내에서 박스의 중심 좌표,

w와 h는 이미지 전체에 대한 상대적인 너비와 높이를 의미한다.

또한 각 셀은 해당 셀에 객체가 있다고 가정했을 때의 조건부 클래스 확률

$\Pr(\text{Class} | \text{Object})$ 도 예측한다.

테스트 시에는 각 바운딩 박스에 대해 클래스 확률과 confidence를 곱하여 최종적으로 클래스별 confidence 점수를 계산하며,

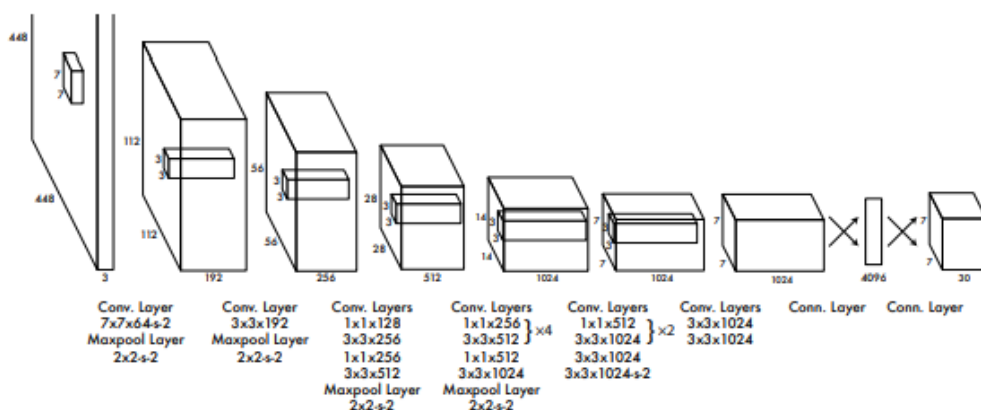
이는 해당 클래스가 그 박스에 존재할 확률과 박스의 정확도를 함께 반영한다.

PASCAL VOC 데이터셋에서는 $S=7$, $B=2$, $C=20$ 으로 설정되며, 최종 출력은 $7 \times 7 \times 30$ 크기의 텐서이다.

2.1 Network Design

초기의 합성곱층은 이미지로부터 특징을 추출하고, FC 층은 출력 가능성을 예측한다.

YOLO의 신경망은 GoogLeNet을 기반으로 하지만, Inception 모듈 대신에 1×1 축소 레이어와 3×3 컨볼루션 레이어를 사용하는 방식으로 구성되어 있다.



아키텍처

전체 네트워크는 24개의 Convolutional Layer와 2개의 FC layer로 구성되어 있다.

속도를 더 높인 Fast YOLO의 경우 Convolutional Layer의 수를 9개로 줄이고 필터 수도 감소시켰다.

이 외의 학습 및 테스트 설정은 동일하다.

2.2 Training

학습은 먼저 ImageNet 1000클래스 분류 데이터셋에서 사전 학습을 수행하고, 이후 객체 탐지를 위한 레이어를 추가한다.

탐지에서는 448×448 크기의 더 높은 해상도를 사용하여 세밀한 시각 정보를 보존하고, 출력 레이어는 바운딩 박스 좌표와 클래스 확률을 동시에 예측한다.

x, y 는 그리드 셀 기준 오프셋으로, w 와 h 는 전체 이미지 기준으로 정규화되며,

$$\phi(x) = \begin{cases} x, & \text{if } x > 0 \\ 0.1x, & \text{otherwise} \end{cases}$$

x 범위에 따른 활성화 함수 적용

마지막 출력층은 선형 활성화 함수를, 나머지 모든 레이어는 Leaky ReLU를 사용한다.

YOLO는 손실 함수로 합성곱 오차를 사용하는데 최적화 하기가 쉬운 장점이 있지만, 실제 객체 탐지의 성능과는 다소 차이가 있다.

특히 많은 셀이 객체를 포함하지 않아 confidence score가 0으로 향하는 경향이 있으며, 이로 인해 학습이 불안정해질 수 있다.

이러한 문제를 해결하기 위해 객체가 있는 셀에 대한 좌표 예측 오류의 가중치를 높이고 ($\lambda_{\text{coord}} = 5$),

객체가 없는 셀의 confidence 오류는 줄인다($\lambda_{\text{noobj}} = 0.5$).

또한, 큰 박스보다 작은 박스의 예측 오차가 IOU에 더 큰 영향을 미치기 때문에, 박스의 너비와 높이를 제곱근 값을 예측하여 상대적 영향을 완화시킨다.

YOLO는 하나의 셀이 2개의 바운딩 박스를 예측하지만, 실제 학습 시에는 IOU 값을 기준으로 각 객체에 가장 적합한 예측자 하나만을 책임있게 지정하여 학습한다.

loss function:

$$\begin{aligned}
 & \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right] \\
 & + \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} \left[\left(\sqrt{w_i} - \sqrt{\hat{w}_i} \right)^2 + \left(\sqrt{h_i} - \sqrt{\hat{h}_i} \right)^2 \right] \\
 & + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 \\
 & + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2 \\
 & + \sum_{i=0}^{S^2} \mathbb{1}_i^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2
 \end{aligned}$$

YOLO 모델의 최종 손실함수

이를 통해 각 바운딩 박스 예측자가 특정 클래스, 크기, 비율에 특화되도록 하여 전반적인 recall을 향상시킨다.

1. 위치 오차 : 바운딩 박스의 중심 (x, y)의 오차
2. 크기 오차 : 박스의 width, weight 오차 (이때, root는 작은 박스의 오차에 더 민감하게 반응하도록 함)
3. 객체 있음에 대한 Confidence 오차 : 박스에 객체가 있을 때 Confidence의 오차
4. 객체 없음에 대한 Confidence 오차 : 객체가 없을 때 값이 0이 되도록 penalize
5. 클래스 확률 오차 : 셀에 객체가 있을 때에만 클래스 확률 예측에 대한 오차

학습은 PASCAL VOC 2007, 2012 학습 및 검증 데이터를 이용하여 약 135 epoch 동안 수행하며,

초기 learning rate는 0.001에서 0.01로 천천히 증가시킨 후 75 epoch 동안 유지시킨다.

이후 0.001로 30 epoch, 0.0001로 다시 30 epoch을 학습한다.

학습 시 batch size는 64, momentum은 0.9, weight decay는 0.0005이며, 과적합 방지를 위해 dropout(rate = 0.5)과 다양한 데이터 증강 기법(스케일링, 이동, 노출 및 채도 조절 등)을 활용한다.

2.3 Inference

YOLO의 추론 과정은 학습과 동일하게 한 번의 신경망 평가만으로 전체 객체 예측이 가능하여 매우 빠르다.

이미지당 98개의 바운딩 박스와 클래스 확률을 예측하며, 여러 셀이 동일 객체를 예측하는 경우가 있을 수 있는데,

이때는 Non-Maximum Suppression을 통해 중복 예측을 제거하고 성능을 추가적으로 개선할 수 있다.

실제로, NMS는 mAP 기준으로 2~3% 향상에 기여한다.

2.4 Limitations of YOLO

하지만 YOLO는 몇 가지 한계를 가진다.

각 셀이 2개의 바운딩 박스만 예측하고 1개의 클래스만 할당할 수 있어 근접한 작은 객체들이 많은 경우 성능이 떨어진다.

또, 새로운 비율이나 배치의 객체에 일반화하는 데 어려움을 겪고, 네트워크 구조상 다운샘플링이 많아 세밀한 특징 추출이 어렵다.

마지막으로, 작은 박스에서의 예측 오차는 IOU에 큰 영향을 주지만, 손실 함수는 이를 동일하게 취급하여 localization 오류가 주요한 성능 저하 요인으로 남는다.

3. Comparison to Other Detection Systems

YOLO는 기존 객체 탐지 기법들의 복잡한 파이프라인을 제거하고, 단일 신경망 구조로 통합하여 빠르고 정확한 객체 탐지를 수행하는 모델이다. 기존의 대표적인 객체 탐지 방법들과 비교하고자 한다.

Deformable parts models

DPM은 슬라이딩 윈도우 방식으로 객체를 탐지하며, 특징 추출, 분류, 박스 예측 등 여러 단계를 분리된 방식으로 수행한다.

반면에 YOLO는 이 모든 단계를 1개의 합성곱 신경망 안에서 동시에 처리하며, 고정된 특징 대신 학습된 특징을 사용하고, 전체 이미지를 고려한 추론을 수행하여 DPM보다 빠르고 정확한 결과를 낸다.

R-CNN

R-CNN 계열의 모델은 슬라이딩 윈도우 대신 Selective Search를 사용해 후보 영역(region proposals)을 만들고, CNN으로 특징을 추출하고, SVM으로 분류하며, 박스 조정 및 중복 제거(NMS)를 수행하는 복잡한 파이프라인을 가진다.

이 과정은 테스트당 40초 이상 소요되는 만큼 느리고, 각 단계가 독립적으로 조정되어야 한다. YOLO는 R-CNN처럼 각 셀마다 후보 박스를 만들고 이를 CNN 특징으로 평가하지만, YOLO는 grid 기반의 공간적 제약을 주어 중복 탐지를 줄이고, 98개의 박스만 예측하여 R-CNN의 2000개보다 훨씬 적으며, 모든 과정을 하나의 네트워크로 통합하여 빠른 속도를 달성한다.

Other Fast Detectors

Fast R-CNN과 Faster R-CNN은 R-CNN의 속도 문제를 개선하기 위해 계산을 공유하고, Region Proposal Network(RPN)를 통해 후보 영역을 생성한다. 하지만 역시나 실시간 처리까지는 성능이 미치지 못한다.

DPM 개선 연구들도 HOG 계산 최적화, GPU 활용을 통해 속도 개선을 시도하였으나, 실시간 성능을 낸 것은 일부에 불과하였다.

YOLO는 아예 파이프라인 구조를 버리고 처음부터 빠른 구조로 설계되었다.

얼굴과 사람을 탐지하는 단일 클래스 탐지기는 클래스 다양성이 적어 최적화가 쉽지만, YOLO는 다중 클래스 탐지를 동시에 수행하는 일반 목적 객체 탐지기로서 더욱 범용적인 모델임을 알 수가 있다.

Deep MultiBox

Deep MultiBox는 CNN을 이용해 관심 영역을 예측하지만, 여전히 후속 분류 단계가 필요하고, 일반 객체 탐지가 불가능하다.

반면 YOLO는 이 모든 과정을 하나의 통합된 시스템으로 처리한다.

OverFeat

OverFeat는 CNN을 활용한 슬라이딩 윈도우 기반 탐지기이며, 지역적 정보만을 기반으로 하기 때문에

전체 문맥을 고려하지 못하고, 많은 후처리 과정이 필요하다.

YOLO는 전체 이미지를 고려하는 구조로 이러한 한계를 극복해낸다.

MultiGrasp

MultiGrasp는 단일 객체의 grasping 위치를 예측하는 구조로, YOLO의 그리드 기반 박스 예측 아이디어에 영감을 준 모델이다.

하지만 단일 객체에서 graspable region만 예측하면 되기 때문에,

다중 객체 탐지와 분류를 동시에 수행하는 YOLO보다 훨씬 단순한 문제를 다룬다.

결론적으로, YOLO는 기존 방법들의 복잡성과 느린 속도를 극복하며, 단일 신경망 기반의 빠르고 정확한 객체 탐지 모델로

설계되었으며, 다중 클래스와 다중 객체에 대해 실시간으로 동작 가능한 범용 탐지기를 지향하는 모델이다.

4. Experiments

YOLO의 성능과 특징을 실험을 통해 분석하고, 기존 객체 탐지 모델들과 비교한 내용을 다루고 있다.

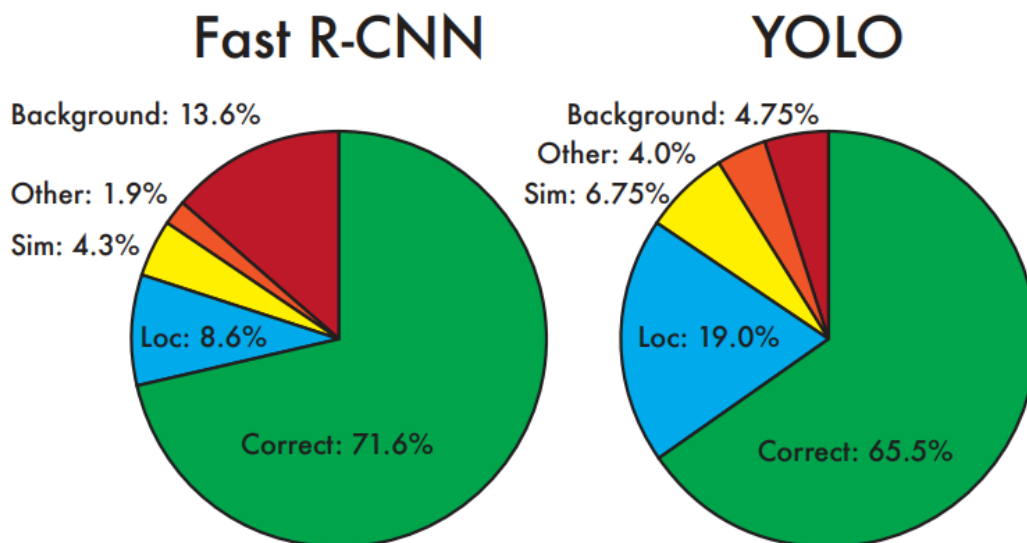
4.1 Comparison to Other Real-Time Systems

YOLO는 PASCAL VOC 2007 데이터셋을 기반으로 다른 실시간 객체 탐지 시스템들과 비교했을 때, 정확도와 속도 면에서 가장 뛰어난 성능을 보였다. 특히 Fast YOLO는 52.7% mAP를 기록하며 기존 실시간 탐지 방법들보다 2배 이상 정확도가 높고, 기본 YOLO 모델은 63.4% mAP를 달성하면서도 실시간 성능(30 FPS 이상)을 유지한다. YOLO를 VGG-16 백본으로 학습시킨 모델은 더 높은 정확도를 얻지만 실시간 속도는 달성하지 못하여 비교용으로만 활용하였다.

비교 대상이 된 Fast R-CNN은 PASCAL에서 가장 빠른 객체 탐지 방법으로, 높은 정확도를 가지고 있지만 Selective Search에 의존하기 때문에 속도가 매우 느리며, 실시간 처리와는 거리가 있다. 반면에 Faster R-CNN은 Selective Search 대신 Region Proposal Network(RPN)을 사용해 속도를 향상시켰지만, YOLO에 비해서는 여전히 느리거나 정확도가 낮은 편이다. Fastest DPM 또한 빠르긴 하나 실시간에는 미치지 못하며 정확도도 상대적으로 낮고, 전반적으로 YOLO는 가장 빠른 속도와 우수한 정확도를 동시에 만족하는 모델로 평가된다.

4.2 VOC 2007 Error Analysis

YOLO와 Fast R-CNN의 오류를 분석한 결과, 두 모델은 서로 다른 유형의 실수를 한다는 점이 밝혀졌다.



Fast R-CNN 과 YOLO의 Error Analysis

각 analysis가 나타내는 부분이다.

1. Correct : correct class and $\text{IOU} > 0.5$
2. Localization : correct class, $0.1 < \text{IOU} < 0.5$
3. Similar : class is similar, $\text{IOU} > 0.1$
4. Other : class is wrong, $\text{IOU} > 0.1$
5. Background : $\text{IOU} < 0.1$ for any object

YOLO는 주로 객체의 위치를 부정확하게 예측하는 Localization Error가 많았고, 반면에 Fast R-CNN은 객체가 아님에도 객체로 잘못 인식하는 Background Error가 많았다. 실제로 Fast R-CNN의 상위 탐지 결과 중 약 13.6%는 아무 객체도 포함하지 않는 배경을 객체로 탐지한 잘못된 예측이었다.

4.3 Combining Fast R-CNN and YOLO

서로 다른 Error 특성을 가진 두 모델을 결합하면 서로를 보완하며 성능 향상이 가능하다는 점에서,
YOLO를 활용해 Fast R-CNN의 배경 오류를 보정하는 방식이 실험되었다.

이러한 결합 방식은 Fast R-CNN이 예측한 각 박스에 대해 YOLO가 유사한 박스를 예측했는지 확인한 뒤,
두 박스의 중첩된 정도와 YOLO의 신뢰도를 바탕으로 해당 예측의 점수를 보정하는 방식으로 이루어진다.

그 결과 Fast R-CNN 단독 모델의 mAP가 71.8%에서 75.0%로 3.2%p 상승하였는데,

	mAP	Combined	Gain
Fast R-CNN	71.8	-	-
Fast R-CNN (2007 data)	66.9	72.4	.6
Fast R-CNN (VGG-M)	59.2	72.4	.6
Fast R-CNN (CaffeNet)	57.1	72.1	.3
YOLO	63.4	75.0	3.2

VOC 2007 데이터셋에 대한 Combination mdoel 실험 결과

이는 Fast R-CNN 모델들끼리의 단순 앙상블에서 얻은 성능 향상인 0.3~0.6%보다도 훨씬 높은 수치이다.

YOLO가 서로 다른 유형의 Error를 하므로 보완 효과가 크다는 것을 보여주는데, 두 모델을 각각 실행하여 결과를 합치므로 속도 측면에서 결합함으로써 얻는 장점이 없다.

4.4 VOC 2012 Results

VOC 2012 test	mAP	aero	bike	bird	boat	bottle	bus	car	cat	chair	cow	table	dog	horse	mbike	person	plant	sheep	sofa	train	tv
MR_CNN_MORE_DATA [11]	73.9	85.5	82.9	76.6	57.8	62.7	79.4	77.2	86.6	55.0	79.1	62.2	87.0	83.4	84.7	78.9	45.3	73.4	65.8	80.3	74.0
HyperNet_VGG	71.4	84.2	78.5	73.6	55.6	53.7	78.7	79.8	87.7	49.6	74.9	52.1	86.0	81.7	83.3	81.8	48.6	73.5	59.4	79.9	65.7
HyperNet_SP	71.3	84.1	78.3	73.3	55.5	53.6	78.6	79.6	87.5	49.5	74.9	52.1	85.6	81.6	83.2	81.6	48.4	73.2	59.3	79.7	65.6
Fast R-CNN + YOLO	70.7	83.4	78.5	73.5	55.8	43.4	79.1	73.1	89.4	49.4	75.5	57.0	87.5	80.9	81.0	74.7	41.8	71.5	68.5	82.1	67.2
MR_CNN_S_CNN [11]	70.7	85.0	79.6	71.5	55.3	57.7	76.0	73.9	84.6	50.5	74.3	61.7	85.5	79.9	81.7	76.4	41.0	69.0	61.2	77.7	72.1
Faster R-CNN [28]	70.4	84.9	79.8	74.3	53.9	49.8	77.5	75.9	88.5	45.6	77.1	55.3	86.9	81.7	80.9	79.6	40.1	72.6	60.9	81.2	61.5
DEEP_ENS_COCO	70.1	84.0	79.4	71.6	51.9	51.1	74.1	72.1	88.6	48.3	73.4	57.8	86.1	80.0	80.7	70.4	46.6	69.6	68.8	75.9	71.4
NoC [29]	68.8	82.8	79.0	71.6	52.3	53.7	74.1	69.0	84.9	46.9	74.3	53.1	85.0	81.3	79.5	72.2	38.9	72.4	59.5	76.7	68.1
Fast R-CNN [14]	68.4	82.3	78.4	70.8	52.3	38.7	77.8	71.6	89.3	44.2	73.0	55.0	87.5	80.5	80.8	72.0	35.1	68.3	65.7	80.4	64.2
UMICH_FGS_STRUCT	66.4	82.9	76.1	64.1	44.6	49.4	70.3	71.2	84.6	42.7	68.6	55.8	82.7	77.1	79.9	68.7	41.4	69.0	60.0	72.0	66.2
NUS_NIN_C2000 [7]	63.8	80.2	73.8	61.9	43.7	43.0	70.3	67.6	80.7	41.9	69.7	51.7	78.2	75.2	76.9	65.1	38.6	68.3	58.0	68.7	63.3
BabyLearning [7]	63.2	78.0	74.2	61.3	45.7	42.7	68.2	66.8	80.2	40.6	70.0	49.8	79.0	74.5	77.9	64.0	35.3	67.9	55.7	68.7	62.6
NUS_NIN	62.4	77.9	73.1	62.6	39.5	43.3	69.1	66.4	78.9	39.1	68.1	50.0	77.2	71.3	76.1	64.7	38.4	66.9	56.2	66.9	62.7
R-CNN VGG BB [13]	62.4	79.6	72.7	61.9	41.2	41.9	65.9	66.4	84.6	38.5	67.2	46.7	82.0	74.8	76.0	65.2	35.6	65.4	54.2	67.4	60.3
R-CNN VGG [13]	59.2	76.8	70.9	56.6	37.5	36.9	62.9	63.6	81.1	35.7	64.3	43.9	80.4	71.6	74.0	60.0	30.8	63.4	52.0	63.5	58.7
YOLO	57.9	77.0	67.2	57.7	38.3	22.7	68.3	55.9	81.4	36.2	60.8	48.5	77.2	72.3	71.3	63.5	28.9	52.2	54.8	73.9	50.8
Feature Edit [33]	56.3	74.6	69.1	54.4	39.1	33.1	65.2	62.7	69.7	30.8	56.0	44.6	70.0	64.4	71.1	60.2	33.3	61.3	46.4	61.7	57.8
R-CNN BB [13]	53.3	71.8	65.8	52.0	34.1	32.6	59.6	60.0	69.8	27.6	52.0	41.7	69.6	61.3	68.3	57.8	29.6	57.8	40.9	59.3	54.1
SDS [16]	50.7	69.7	58.4	48.5	28.3	28.8	61.3	57.5	70.8	24.1	50.7	35.9	64.9	59.1	65.8	57.1	26.0	58.8	38.6	58.9	50.7
R-CNN [13]	49.6	68.1	63.8	46.1	29.4	27.9	56.6	57.0	65.9	26.5	48.7	39.5	66.2	57.3	65.4	53.2	26.2	54.5	38.1	50.6	51.6

PASCAL VOC 2012 데이터셋에 대한 각 모델의 수치 분석

VOC 2012 데이터셋에서도 YOLO는 57.9%의 mAP를 기록하며 기존 R-CNN 계열과 유사한 수준의 성능을 보여주었고,

일부 소형 객체(예: bottle, sheep, tv/monitor 등)에서는 성능이 떨어졌으나, cat, train 등의 카테고리에서는 더 높은 성능을 기록하였다.

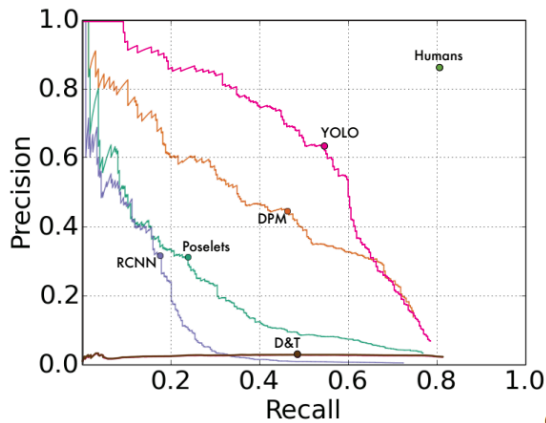
YOLO와 Fast R-CNN을 결합한 모델은 성능이 2.3% 정도가 향상되어 공공 리더보드에서 5 단계가 상승한 순위를 기록하였다.

4.5 Generalizability: Person Detection in Artwork

마지막으로 YOLO의 일반화 성능을 평가하기 위해 artwork 이미지에서 사람을 탐지하는 실험도 수행되었다.

자연 이미지에서 학습된 모델들이 artwork 이미지에서는 성능이 크게 떨어지는 경향이 있는데,

YOLO는 상대적으로 이러한 환경 변화에도 강한 성능을 유지하였다.



(a) Picasso Dataset precision-recall curves.

	VOC 2007 AP	Picasso AP	Picasso Best F_1	People-Art AP
YOLO	59.2	53.3	0.590	45
R-CNN	54.2	10.4	0.226	26
DPM	43.2	37.8	0.458	32
Poselets [2]	36.5	17.8	0.271	
D&T [4]	-	1.9	0.051	

(b) Quantitative results on the VOC 2007, Picasso, and People-Art Datasets. The Picasso Dataset evaluates on both AP and best F_1 score.

Figure 5: Generalization results on Picasso and People-Art datasets.

R-CNN의 경우 분류 시 작은 영역들만 보고, Selective Search가 자연 이미지에 맞춰 설계되어 있어

예술 이미지에서는 성능이 크게 저하되었다.

DPM의 경우 artwork 적용 시에도 성능이 R-CNN처럼 확 떨어지지 않는으나, 애초에 낮은 AP값에서 시작한다.

YOLO는 DPM과 유사하게 객체의 크기, 모양, 위치 관계를 함께 학습하기 때문에, artwork와 자연 이미지가 물론 픽셀 단에서는 다르지만, 객체의 크기와 모양 측면에서는 유사하기 때문에 예술 이미지에서도 객체를 잘 탐지할 수 있다.

이와 같이 YOLO는 정확도, 속도, 오류 유형, 일반화 성능 측면에서 기존 방법들과 뚜렷한 차별성을 보이며,

단일 네트워크 구조를 통해 실용적이고 효율적인 객체 탐지 시스템을 다양한 실험을 통해 입증하였다.

5. Real-Time Detection In The Wild

YOLO는 빠르고 정확한 객체 탐지 모델로, 실시간 컴퓨터 비전 응용에 매우 적합하다. YOLO를 웹캠에 연결하는 경우, 이미지 캡처부터 객체 탐지 결과를 화면에 표시하는 전체 과정이 실시간으로 이루어지게 된다.

이미지 하나하나를 각각 독립적으로 처리하지만, 웹캠과 함께 사용할 경우 움직이는 객체를 지속적으로 탐지하면서 추적 시스템처럼 동작한다.

이러한 시스템은 직관적이고 사용자와의 상호작용도 뛰어나다.

6. Conclusion

YOLO는 객체 탐지를 위한 통합 모델로, 복잡한 구성 없이 전체 이미지를 대상으로 직접 학습이 가능하다.

기존의 분류 기반 접근 방식과 달리, YOLO는 객체 탐지 성능과 직접적으로 연결된 손실 함수로 학습되며, 전체 네트워크가 함께 학습이 된다.

특히 Fast YOLO는 기존의 존재하던 방식들 중에서도 가장 빠른 범용 객체 탐지기로, YOLO는 실시간 객체 탐지의 새로운 기준을 제시하며, 새로운 환경에도 일반화 성능이 뛰어나다.

이로 인해 빠르고 견고한 객체 탐지가 필요한 다양한 응용 분야에 적합한 모델로 평가받기도 한다.

공감

'카테고리 없음'의 다른글

이전글 [Euron] 12Week_ BLIP : Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation